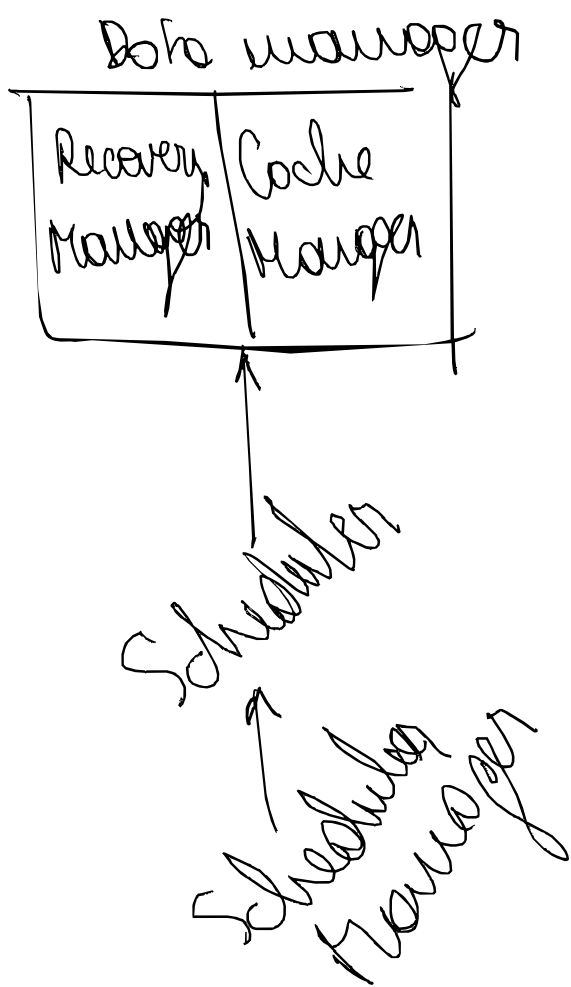




Problemas de control de concurrencia

- lost update
- dirty read
- non-repeatable read.

Secuencia de ejecución correcta

→ $\Leftrightarrow$ la ejecución es coherente con alguna posible serialización de los transacciones

T1 ; T2

T2 ; T1.

(que NO ABORTAN)

Scheduler

- opening (procesa primero)
- conservador

problema

- richazgar
- procesar
- demorar

Ctrl de conc.

LOCK BINARIO - SHARED LOCK

- mientras operen en distintos registros $\Rightarrow$ NO HAY PROBLEMA
- si ocurren el mismo, lockeo y hace esperar al resto hasta que libere.
- problemas
 - deadlock
 - no hace falta lockeo en lectores

read $\rightarrow$ shared lock (muchos pueden leer)
write $\rightarrow$ exclusive lock

TWO PHASE LOCKING (2PL)

- más permisivos
- intenta liberar los locks que se puedan antes
- fase de crecimiento (mayor locks)
- fase de decrecimiento (liberar locks)
- por cada transacción debe ser crecimiento y decrecimiento.
- consecutivo en la historia de ejecución.

→ Timeout $\rightarrow$ eficiente p/ evitar deadlocks (si es deadlock $\rightarrow$ lo suelta)

→ Wait For Graph (WFG)

→ indicar qué tx/modos están esperando por otros.

TIMESTAMPS

→ voy anotando quién tomó el registro y me fijo de número los accesos según algún índice / timestamp

→ que se mantenga una ejecución ordenada x timestamp $\Rightarrow$ si se rompe el orden, aborta.

→ también diferencia x R o W

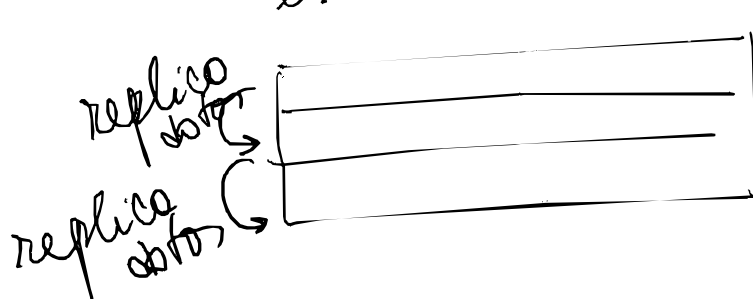
→ MARCA: YA NO USÉ, no lockeo.

(si el índice / timestamp es menor al último que usó el registro, se rompe)

MULTIVERSION

→ en función de quién leyó y escribió nuevos valores mínimos y máximos de timestamp p/ que puedan leer o reutilizarlos que tuvieron antes

→ guarda los datos de un registro completo junto con los "timestamps" para los que el registro es válido.



MATA A LOS QUE SE APURARON

PHANTOM PROBLEM (otra problema de concurrencia)

NIVELES DE AISLAMIENTO SQL